

🌟 Member-only story

TOON vs JSON: The Modern Data Format Showdown Built for the AI Era

3 min read · Nov 9, 2025



Mudaser Ali

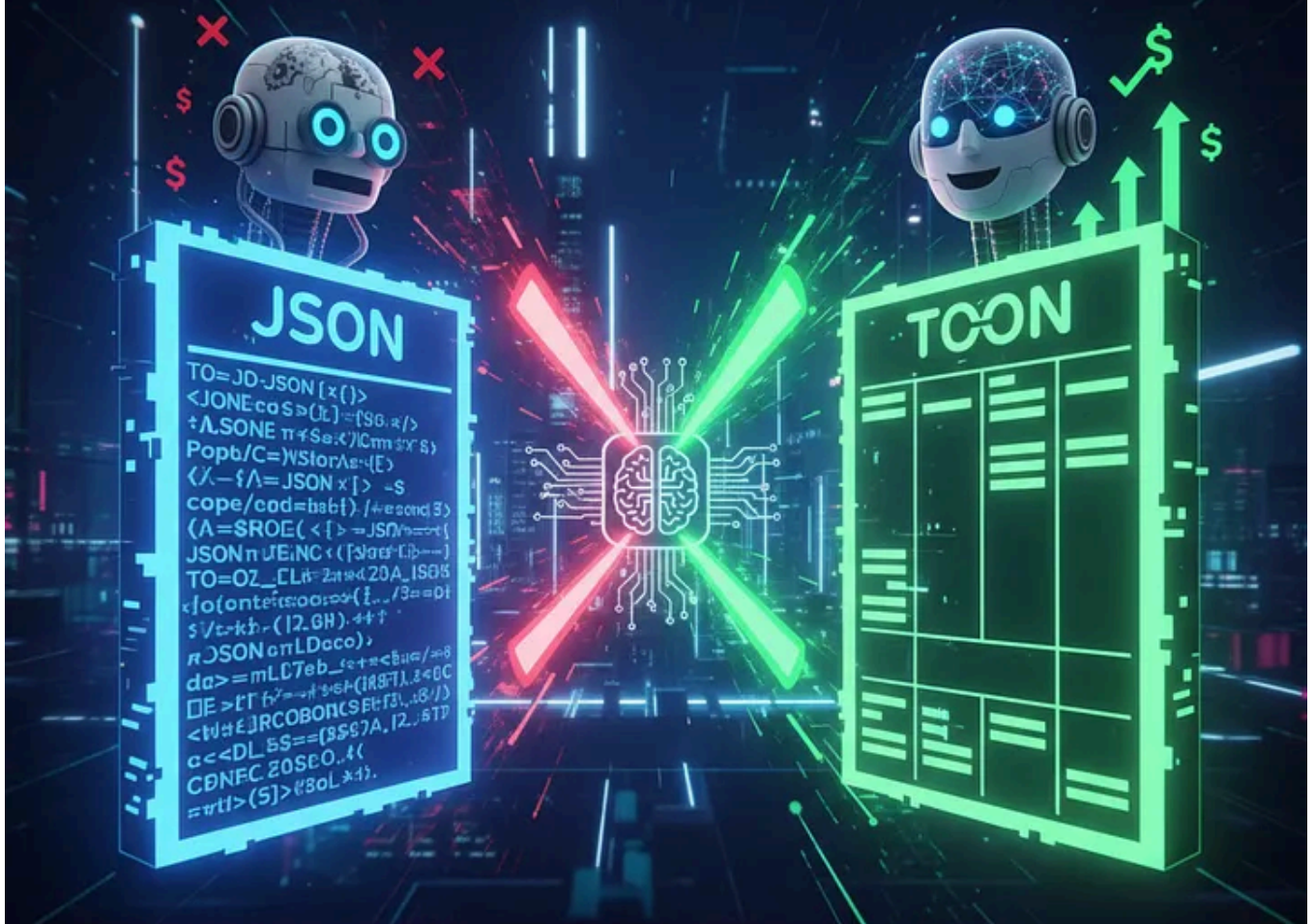
Follow

▶ Listen

📄 Share

⋮ More

TOON vs JSON: The Modern Data Format Showdown Built for the AI Era 🤖



The way we structure and exchange data is evolving faster than ever, driven by the rise of Large Language Models (LLMs). While **JSON (JavaScript Object Notation)** has been the undisputed champion for over a decade, a new, purpose-built format called **TOON (Token-Oriented Object Notation)** is emerging as the preferred choice for GenAI developers.

If your work involves LLMs, token costs, and structured data, this showdown is for you.

. . .

1. JSON: The Universal Workhorse

For years, JSON has been the lingua franca of the web. Its lightweight, human-readable structure, based on explicit delimiters, made it perfect for general-purpose data exchange.

- **Syntax Style:** Verbose; relies heavily on repetitive keys, curly braces (`{ }`), square brackets (`[ ]`), colons (`:`), and commas (`,`).
- **Key Benefit: Universal Compatibility** across all languages and systems.
- **Primary Drawback for AI: High Token Count.** Every piece of punctuation and repeated key consumes a token, inflating costs and context window usage in LLM calls.

Simple JSON Example (Uniform Data)

Here is a typical JSON array containing three user records:

```
{
  "users": [
    { "id": 1, "name": "Sreeni", "role": "admin" },
    { "id": 2, "name": "Krishna", "role": "admin" },
    { "id": 3, "name": "Aaron", "role": "user" }
  ]
}
```

2. TOON: Built for Token Efficiency

TOON (Token-Oriented Object Notation) is a next-generation format specifically engineered to solve the token efficiency problem in LLM workflows. It achieves dramatic token savings by minimizing redundant syntax.

- **Syntax Style:** Compact; uses **indentation** and explicitly defined **column headers** just once for an entire block of data.
- **Key Benefit: Extreme Token Efficiency** (30–60% savings on tabular data), translating directly to cheaper and faster LLM calls.
- **Design Rationale:** Declaring the structure once (in the header) and then streaming the values (in tabular rows) eliminates the token cost of repeating keys and punctuation.

Simple TOON Example (Same Uniform Data)

Here is the exact same data represented in TOON:

```
users[3]{id,name,role}:  
1,Sreeni,admin  
2,Krishna,admin  
3,Aaron,user
```

Savings Highlight: For this type of uniform, tabular data, TOON offers roughly **50% fewer tokens** than JSON. This reduction scales significantly with larger datasets.

3. The Trade-Off: Complex Nested Objects 🤔

While TOON shines with uniform arrays, the comparison changes when dealing with highly nested, irregular structures.

✎ Complex Nested JSON Example

```
{  
  "project": {  
    "name": "Apollo",  
    "status": "active",  
    "team": [  
      {  
        "id": 101,  
        "role": "lead",  
        "contact": { "email": "alice@ex.com" }  
      }  
    ]  
  }  
}
```

JSON Advantage: The explicit use of `{}` and `[]` provides clear, **unambiguous boundaries** for every element, regardless of nesting depth. This ensures robust and reliable parsing by LLMs, which is critical for complex logic or configuration files.

✎ Complex Nested TOON Example (Same Data)

```
project:  
  name: Apollo  
  status: active
```

```
team[1]{id,role,contact}:  
  101,lead,contact{email}:  
    alice@ex.com
```

TOON Consideration: TOON handles nesting via indentation. For arrays (`team`), it still saves tokens. However, for the inner, single-level `contact` object, the token-saving benefit is minimal. In fact, for very deep and irregular nesting, the token count difference between TOON and a highly compacted JSON string may shrink or occasionally reverse.

. . .

Conclusion: When to Choose Which

The future of data formats is not one or the other, but knowing when to use each one strategically:

- Use TOON when **token efficiency** and large, **uniform data** are involved. This is ideal for RAG pipelines, sending database query results to an LLM agent, or structured LLM output generation.
- Use JSON when **interoperability** and **parsing reliability** for **deeply nested, non-uniform structures** are paramount. This is still the standard for API definitions and complex configurations.

TOON is the specialized, cost-saving tool for the AI developer, designed to optimize the one resource that matters most in LLM-driven applications: the token count.

Are you ready to start bench-marking your LLM costs using TOON, or would you like to see a more detailed breakdown of the libraries available to convert JSON to TOON?

JavaScript

Json

Nodejs



Follow

Written by Mudaser Ali

36 followers · 8 following

{ name: Mudaser Ali aka SMAK, skills: NoBody is pErfect i M nObOdy, passion: {love: [programming, photography]} }

Responses (2)



Bgerby

What are your thoughts?



Nickolay Vasiliev

4 days ago



Looks like the TOON loses its beauty when it tries to be a YAML getting nested :)



8

[Reply](#)



Ali Farhat

4 days ago



Hi there, great article, would you mind adding a link to our tool in your article?

<https://scalevise.com/json-toon-converter>

Thanks in advance! 🙌🙌



2

[Reply](#)

More from Mudaser Ali



Mudaser Ali

SAML SSO Identity Provider in Node.js: The Complete Guide (Part 1)

Building Enterprise Authentication in 5 Minutes



Aug 9



1



 Mudaser Ali

SAML SSO Service Provider in Node.js: Complete Implementation Guide (Part 2)

Consuming SAML Assertions Like a Pro

★ Aug 16



 Mudaser Ali

React Native Sticky Date like whatsapp using Inverted SectionList

If you are currently developing a chat application and you have a requirement to stick a date while scrolling messages to let user know...

★ May 10, 2020  27  1



 Mudaser Ali

The Era of Complex RegEx for URLs Is Over: Welcome the Baseline URLPattern API

For years, URL parsing meant writing fragile regular expressions or relying on external libraries. That ends now. The URLPattern API is...

Nov 3  1



See all from Mudaser Ali

Recommended from Medium

 In Medialesson by Marius Schröder

JSON vs TOON—A new era of structured input?

Why structure matters more than ever

Nov 3  797  42



 CodeOrbit

I Rebuilt My RAG Pipeline 11 Times—Here's the Architecture That Finally Scaled

How I went from \$8,400/month cloud costs and 18-second queries to \$1,240/month and 420ms response times after 11 failed attempts.



Nov 9



217



7



In Towards AI by Kushal Banda

From JSON to TOON: Evolving Serialization for LLMs

When you're scaling AI applications, token efficiency isn't just a buzzword it's your bottom line. Every token wasted is money left on the...

Nov 9



20



2



İsmail Kağan Acar

TOON: New data format for LLM Applications

İsmail Kağan Acar

Nov 7  73  1



 In AI Software Engineer by Joe Njenga

Anthropic Just Solved AI Agent Bloat—150K Tokens Down to 2K (Code Execution With MCP)

Anthropic just released smartest way to build scalable AI agents, cutting token use by 98%, shift from tool calling to MCP code execution

 Nov 6  655  48



 Ramesh Kannan s

TOON: A New Input Format to Make AI Understand You Faster

We are living in a time where talking to AI has become as common as sending a text. We ask AI to write emails, generate workout plans...

★ Nov 9 🤝 35 💬 1



See more recommendations